Análise Exploratória de Dados

Capítulo 12 — Streamlit e Llama

Cayo Oliveira

Sumário

12 Dashboard analítico com Streamlit e API de LLM **3**
12.1 Separar interface, domínio, dados e adaptador LLM 3
12.2 Construir o painel com gráfico e denominador explícito 5
12.3 Gerar narrativa com contexto mínimo e validação 6
12.4 Estado, cache e falhas de rerun 9
12.5 Roteiro da aula e decisão de publicação 10
12.6 Implementar o adaptador e a fronteira de validação 10

Capítulo 12

Dashboard analítico com Streamlit e API de LLM

O notebook da aula responde à pergunta, mas um colega não consegue reproduzir filtros, ver versão da fonte nem distinguir texto gerado de estatística calculada. A equipe transforma a análise em uma aplicação: carrega dados, filtra, calcula, desenha gráfico e usa LLM apenas para explicar resultados aprovados.

Streamlit permite construir interface Python para exploração e dashboard. O rerun do script exige atenção a estado e efeitos repetidos; cache exige chave que inclua os dados; conexão com modelo deve ficar no servidor e não no navegador. A aplicação é pequena, mas já possui camadas e riscos de sistema.

O fluxo é: entrada e identidade, seleção segura de dados, resumo agregado, função analítica determinística, visualização, geração textual opcional, validação e resposta. O modelo nunca recebe a planilha integral se o insight puder ser explicado com estatísticas agregadas e dados minimizados.

O cenário de aula compara taxa de aprovação por curso e período, mantendo amostras pequenas visíveis. A squad constrói um dashboard em que pergunta, gráfico e narrativa compartilham filtro e atualização. O primeiro caminho conecta a API OpenAI pelo backend, com a chave fornecida pelo professor e guardada localmente como segredo. Um modelo aberto local permanece como alternativa de comparação, mas disponibilidade de hardware, licença e custo operacional precisam ser examinados.

12.1 Separar interface, domínio, dados e adaptador LLM

A interface apresenta controles e resultados, mas não concentra regras de cálculo. Um módulo de domínio calcula métricas, um repositório lê arquivos ou banco, um adaptador chama modelo e um serviço coordena validação. Essa divisão permite testar cálculo sem abrir interface e trocar fornecedor sem reescrever a EDA.

O DataFrame possui regras de acesso próprias. Em aplicação didática carregamos CSV sintético; em ambiente real identificamos fonte, proprietário, período, grão e campos sensíveis. A coluna que não participa da pergunta não entra no prompt ou no trace.

Adaptador responde a um contrato interno: recebe pergunta e resumo aprovado, devolve rascunho associado à versão e ao modelo. Uma API hospedada e um runtime local podem compartilhar a mesma interface interna, mas diferem em processamento, infraes-

trutura, privacidade, disponibilidade, licença e manutenção. O cliente não assume que local significa automaticamente seguro, nem que API hospedada pode receber qualquer dado.

Um mock permite construir e testar interface antes de configurar provedor. O mock não mede qualidade de geração; ele valida transições, estado sem evidência, resposta inválida e rota de erro. Avaliação com modelo real ocorre somente com autorização e conjunto permitido.

Os testes unitários ficam no serviço de domínio e validam agregação e proporções. Testes de integração cobrem leitura de arquivo e fluxo de interface. Essa separação ajuda o estudante a descobrir se um erro veio de cálculo, estado de tela, transporte ou geração.

Uma arquitetura mínima ainda precisa separar responsabilidades. A interface recolhe pergunta, filtros e ações; o serviço de aplicação coordena o fluxo; o domínio define métricas, regras e formatos; o repositório acessa os dados; o adaptador de LLM transforma mensagens e recebe saída; e o validador decide se a saída pode aparecer. Essa divisão permite substituir o fornecedor ou desligar geração sem reescrever cálculo e visualização.

O fluxo de uma solicitação começa com validação de entrada e política. A aplicação obtém identidade e finalidade, normaliza a intenção, calcula agregado permitido, constrói pacote mínimo de evidências e chama o modelo com instruções de tarefa. A resposta é parseada em estrutura conhecida, confere referências e números e só então passa à tela. Erro de rede, JSON inválido, timeout ou resposta vazia levam a fallback seguro e mensagem clara.

Uma mensagem do tipo user contém pedido e contexto autorizado, mas não deve transportar instruções privilegiadas ou dados desnecessários. Instruções de aplicação ficam no lado de sistema/desenvolvedor conforme a API e devem dizer papel, limites, formato e recusa. A resposta assistant é rascunho sujeito à validação; uma chamada de ferramenta retorna como resultado tipado. O sistema não mistura mensagem apresentada com execução já autorizada.

O segredo de API existe no ambiente de servidor, no secret store ou na configuração local ignorada pelo controle de versão. Ele nunca entra no repositório, no bundle do navegador, em logs, capturas de tela ou mensagens enviadas ao modelo. O serviço limita gasto, frequência e tamanho de prompt; troca e revoga a chave quando necessário. Um mock possibilita que todos desenvolvam interface sem expor uma credencial compartilhada.

Dependências têm versões compatíveis e devem ser isoladas em ambiente virtual. O arquivo de configuração lista Streamlit, pandas, biblioteca gráfica e SDK de provedor; o README documenta execução e variáveis necessárias. Versões travadas reduzem diferenças inesperadas. Mesmo assim, alterações de API ou modelo exigem uma revisão do adaptador e da suíte de avaliação antes de atualizar dependências.

A aplicação mínima pode trabalhar com CSV didático, mas define contrato de entrada: colunas exigidas, tipos, valores aceitos e validação de duplicatas. Erro de arquivo inválido não vira falha do modelo. Um módulo de carregamento valida o schema e retorna dado em forma canônica; a análise atua sobre essa forma, deixando fonte e transformação rastreáveis.

A camada de interface precisa permanecer utilizável sem o provedor. Quando a chamada falha, ainda apresenta tabela, gráfico e indicadores determinísticos e informa que a narrativa assistida está indisponível. Esse fallback mantém o objetivo pedagógico, reduz dependência operacional e torna possível comparar o valor incremental da LLM com um baseline simples.

12.2 Construir o painel com gráfico e denominador explícito

O gráfico é gerado a partir de função analítica reproduzível. Filtros modificam recorte, tabela, título e resumo; não podem atualizar apenas o texto. O dashboard mostra tamanho do grupo e intervalo de tempo junto à taxa, para evitar que o número pareça universal.

Código da squad — aplicação Streamlit com análise local

```
# Importa Streamlit para montar controles e componentes da aplicação.
import streamlit as st
# Importa pandas para carregar, filtrar e agregar dados tabulares.
import pandas as pd
# Importa matplotlib para criar uma visualização local controlada.
import matplotlib.pyplot as plt
# Define título e contexto para quem abrir a aplicação.
st.title("Painel de apoio acadêmico --- dados sintéticos")
# Cria uma tabela pequena e fictícia para o exercício.
dados = pd.DataFrame({"curso": ["SI", "SI", "ADS", "ADS"],
                      "periodo": ["2026.1", "2026.2", "2026.1", "2026.2"],
                      "matriculas": [80, 90, 60, 70],
                      "aprovados": [60, 63, 42, 49]})
# Apresenta filtro cujas opções vêm da própria tabela.
periodo = st.selectbox("Período", sorted(dados["periodo"].unique()))
# Seleciona o subconjunto correspondente ao filtro informado.
recorte = dados.loc[dados["periodo"].eq(periodo)].copy()
# Calcula proporção usando numerador e denominador visíveis.
recorte["taxa"] = recorte["aprovados"] / recorte["matriculas"]
# Exibe contagens para evitar interpretação sem suporte amostral.
st.dataframe(recorte[["curso", "aprovados", "matriculas", "taxa"]])
# Cria gráfico de comparação para os cursos no período escolhido.
fig, ax = plt.subplots()
# Desenha barras com taxa, não com volume bruto.
ax.bar(recorte["curso"], recorte["taxa"], color="#2463A6")
# Rotula a unidade e explicita o recorte temporal.
ax.set_ylabel("Aprovados / matrículas")
# Informa no título que o dado é sintético.
ax.set_title(f"Taxa observada em {periodo} --- sintético")
# Apresenta o gráfico ao lado da tabela de suporte.
st.pyplot(fig)
```

O app acima é executável com dependências instaladas e demonstra filtros e gráficos, não autenticação institucional. Um sistema real valida schema ao carregar, trata ausência, registra atualização e restringe linha por identidade no backend.

Uma squad adiciona comparação com o período anterior e um alerta de grupo pequeno. Em vez de esconder curso sem base suficiente, o componente mostra dados insuficientes e não solicita ao LLM que complete valor ausente.

O grupo muda o período e confere se texto e visual mudam juntos. Depois compara uma taxa com denominador distinto. Se a diferença for grande, as equipes apresentam contagens, não apenas a altura das barras, e investigam composição por coorte.

O painel deriva todos os componentes de um mesmo estado validado. A escolha de período filtra o conjunto uma única vez; os indicadores, contagens, tabela, figura e resumo recebem o mesmo objeto de resultado. Evitamos recalcular partes em callbacks separados, pois filtros divergentes geram telas que parecem consistentes, mas descrevem populações distintas. O estado inclui identificador da fonte e horário de atualização.

Uma métrica deve ser implementada como função pura: recebe dado e filtros, devolve valores e metadados, sem alterar sessão ou chamar rede. Essa forma permite reproduzir um exemplo pequeno e comparar resultados exatos. Por exemplo, a taxa de aprovação recebe contagem de matrícula elegível e aprovadas, rejeita denominador zero e retorna taxa como razão; a interface escolhe formato percentual sem mudar o cálculo.

Gráficos respeitam tipo e cardinalidade. Poucas categorias podem ser comparadas com barras; muitos cursos pedem ordenação e talvez busca; série temporal demanda períodos consistentes; distribuição de notas pode requerer histograma e boxplot. A aplicação não cria eixos arbitrários a partir de instruções textuais do modelo. A função visual conhece unidade, escala e limites, e rótulos explicam se o valor representa contagem, proporção ou diferença.

O tamanho dos grupos acompanha as taxas. Para 2/3 e 1/3, mostrar apenas 66,7

A função de gráfico também deve lidar com acessibilidade e exportação. Paleta possui contraste suficiente e não usa apenas cor; eixos têm unidades, títulos são descritivos, ordem acompanha a pergunta e a tabela pode ser lida por tecnologia assistiva. Uma captura exportada preserva filtros e data. Um gráfico que funciona apenas em tela ampla ou hover do mouse ainda não atende a todos os estudantes.

A aplicação controla estado de sessão com cuidado. Streamlit reexecuta o script após interações; objetos temporários são recalculados e estado necessário fica em session state com chaves explícitas. O cache só armazena transformação determinística e inclui fonte, versão, filtros e identidade quando aplicável. Uma chamada de modelo não é cacheada entre usuários se contexto ou permissão diferirem.

Para verificar consistência, a equipe altera um filtro e prevê quais componentes devem mudar. Se período muda, a consulta, contagem, título, evidência e narrativa são invalidados; se apenas o tipo de visual muda, o agregado permanece, mas a figura muda. Esse mapa de dependências ajuda a encontrar bugs de estado e explica por que cache sem chave suficiente pode servir um insight velho.

12.3 Gerar narrativa com contexto mínimo e validação

Antes de chamar o modelo, a aplicação calcula resumo com métricas, filtros, data e avisos de tamanho. O LLM recebe um contrato de linguagem: descrever associação, não afirmar causa; não criar número; citar qual recorte recebeu; e apontar quando a amostra é pequena.

O texto continua sendo rascunho. Um validador confere se os números mencionados existem no resumo, se período confere, se há fonte e se frases de causalidade não extrapolam. Regras automáticas detectam parte dos defeitos; revisão humana é necessária em narrativa relevante.

Código da squad — adaptador mock e verificação de número

```
# Calcula um resumo agregado que não contém identificadores pessoais.
resumo = {"periodo": periodo, "cursos": recorte["curso"].tolist(),
```

```
          "taxas": recorte["taxa"].round(3).tolist(),
          "n": recorte["matriculas"].tolist()}
# Define adaptador de demonstração que funciona sem serviço externo.
def gerar_narrativa_mock(resumo_aprovado):
    # Constrói rascunho apenas com informação do resumo fornecido.
    return (
        f"No período {resumo_aprovado['periodo']}, compare as taxas; "
        "confira os denominadores antes de decidir."
    )
# Gera texto no modo local controlado.
rascunho = gerar_narrativa_mock(resumo)
# Exige que o período que aparece no rascunho esteja no resumo.
assert resumo["periodo"] in rascunho
# Marca a saída como narrativa candidata, não insight confirmado.
st.caption(
    "Rascunho automatizado; valide números e contexto "
    "antes de compartilhar."
)
# Exibe o texto sem ocultar seu estado de revisão.
st.write(rascunho)
```

Uma conexão real à OpenAI ou a um servidor local substitui o adaptador, não o cálculo. A chave fica no servidor; conteúdo transmitido é minimizado; nome e versão do modelo são registrados. O exemplo didático a seguir mostra o fluxo essencial com a Responses API; os valores já chegam calculados e a resposta continua sob validação.

```
from openai import OpenAI
import json

cliente = OpenAI()  # lê OPENAI_API_KEY do ambiente do servidor
resumo = {
    "periodo": periodo,
    "taxas": recorte[["curso", "taxa"]].to_dict("records"),
    "evidence_ids": ["taxa_por_curso_v1"],
}
resposta = cliente.responses.create(
    model="gpt-5-mini",
    instructions=(
        "Escreva uma leitura breve usando somente as evidências. "
        "Não alegue causalidade; inclua o ID da evidência usada."
    ),
    input=json.dumps({"pergunta": pergunta, "evidencias": resumo}),
)
rascunho = resposta.output_text
```

O trecho é um ponto de partida e não demonstra saída estruturada ou segurança completa. Em uma versão robusta, a aplicação exige schema de saída, valida IDs contra a lista local, trata exceções específicas, limita tamanho e custo, e mostra erro legível quando a resposta não passa. A chamada ocorre após ação explícita do usuário, não em cada rerun do Streamlit.

Para a aula, o professor distribui a chave de forma controlada e os alunos a configuram no ambiente próprio; a chave não entra no repositório nem em código cliente. A conta e os limites de uso permanecem sob responsabilidade definida pelo professor. O protótipo funciona sem serviço externo, usando mock, para que nenhum grupo seja bloqueado por conexão, saldo ou disponibilidade.

Modelo local requer atualização, monitoração, capacidade de hardware e revisão da licença específica. A escolha entre local e API deve comparar confidencialidade, qualidade no idioma e tarefa, latência, custo previsível, limite de uso e facilidade de desligar. Não existe uma opção universalmente gratuita e sem custo de operação.

O contraexemplo é pedir ao LLM para recalcular a proporção a partir de uma tabela extensa e aceitar o número produzido. Cálculo fica em pandas ou SQL testado; linguagem natural pode resumir a evidência já calculada. Essa divisão reduz erro aritmético e torna resultado reproduzível.

O modelo recebe um resumo agregado, não a tabela linha a linha. Um pacote de contexto pode conter pergunta normalizada, período, filtros, definição da métrica, numeradores, denominadores, valores e caveats. O backend exclui colunas identificadoras e campos que não participam da tarefa. Minimização reduz exposição, custo e distração e facilita conferir as alegações contra um conjunto compacto.

A instrução de geração define papel e limites: escrever um resumo para público específico; usar somente evidências fornecidas; separar observação de hipótese; não inventar causalidade; citar IDs; pedir esclarecimento quando falta informação; devolver campos estruturados. A instrução não substitui validação no código. O sistema trata o conteúdo recuperado ou fornecido pelo usuário como dado, não como permissão para ignorar políticas.

Saída estruturada simplifica a revisão. Em vez de texto arbitrário, a resposta pode incluir headline, observations, caveats, next_check e evidence_ids. O parser verifica schema, tamanho, enumerações e referências. O renderer formata elementos validados. Se um ID não existe, a frase é retida ou a aplicação pede nova geração com feedback controlado; não apresenta citação inventada como evidência.

A validação numérica pode extrair porcentagens e compará-las aos fatos calculados, mas valores arredondados requerem tolerância definida. Se a evidência registra 0,667 e a narrativa diz 76

O prompt precisa ter orçamento de contexto. Enviamos apenas definições necessárias e top resultados, mantendo evidência completa no aplicativo. Um resumo longo pode introduzir conflito e encarecer a chamada; um resumo curto demais pode omitir caveat importante. A equipe mede tokens aproximados e usa limite de saída. Se exceder, reduz documentos ou solicita uma tarefa menor em vez de truncar silenciosamente uma definição.

Uma experiência útil explica fonte e limite junto da narrativa. A resposta mostra nome e versão do conjunto, data de atualização, definição da métrica e IDs que abrem tabela ou documento. A pessoa pode alternar para somente fatos e inspecionar valores. Transparência não significa expor prompt interno ou credencial; significa tornar análise e origem verificáveis.

Avaliamos a narrativa com exemplos de sucesso e falha. Sucesso: resumo cita evidência correta e qualifica amostra pequena. Falha: atribui mudança a uma política sem desenho causal; inventa grupo ausente; transforma diferença descritiva em recomendação. Revisores identificam a sentença específica e classificam o erro, gerando material para melhorar prompt, schema, cálculo ou política, conforme a origem.

Se a API estiver indisponível, a interface oferece resposta de fallback construída por

template a partir dos mesmos fatos, ou omite narrativa e mantém a análise. A escolha depende da qualidade e do risco do template. O produto informa qual caminho produziu a saída, permitindo comparar texto determinístico e texto gerado na apresentação dos estudantes.

12.4 Estado, cache e falhas de rerun

Streamlit reexecuta o script após interação. Uma chamada ao LLM no nível global pode ocorrer novamente ao mover um filtro, gerar custo e duplicar logs. Um botão de envio, chave de idempotência e estado de sessão controlam a ação; cache só é aplicado quando saída pode ser compartilhada com segurança.

Cache inclui hash da fonte, filtros, métrica, prompt e versão de modelo. Se qualquer insumo muda, a resposta é invalidada. Não guardar saída sensível em cache global sem escopo de usuário. A equipe testa dois perfis e diferentes períodos para detectar reutilização cruzada.

Timeout, limite de taxa, serviço offline e JSON inválido produzem estados visíveis. O painel mantém gráficos determinísticos mesmo quando narrativa falha, mostra fonte e não inventa explicação substituta. A experiência degradada é planejada.

O time calcula quantas chamadas um usuário pode disparar e define limite de tamanho, repetição e volume por período. Um cache compartilhado é revisto quanto à identidade; cache de sessão pessoal não pode ser reutilizado como dado global.

Um teste de caos didático desliga o adaptador durante uma interação. Os estudantes verificam que painel mantém gráfico e informa indisponibilidade textual, sem apagar filtro ou duplicar cobrança. A recuperação é registrada no log mínimo.

Uma sessão possui estado de interface, estado analítico e estado de geração, que não devem ser confundidos. Seleção de filtros pertence à interface; consulta e resultado pertencem à análise; rascunho e avaliação pertencem à geração. Cada alteração invalida apenas derivados dependentes, preservando escolhas válidas. Esse modelo reduz reexecução desnecessária e dificulta que texto de uma consulta antiga sobreviva a filtros novos.

Cache de leitura é adequado para arquivo estático ou consulta determinística com chave completa. A chave pode incluir hash da fonte, versão semântica, filtros normalizados e usuário/política quando a resposta for restrita. A chamada à LLM pode não ser cacheável se a saída depende do modelo, instrução, evidências ou perfil. Reuso entre identidades é proibido quando o conteúdo ou acesso diferem.

Timeout e retry precisam de orçamento. Uma solicitação pode levar mais que o tempo de interface; duas retentativas automáticas multiplicam custo e duplicam ações se a ferramenta não for idempotente. Para uma tarefa apenas narrativa, retry limitado pode ser seguro; para função que altera estado externo, exigimos chave de idempotência e confirmação. No protótipo não há ação externa, e essa fronteira é mantida deliberadamente.

Erros recebem categorias: arquivo inválido, falta de coluna, filtro sem resultado, cálculo indefinido, falha de rede, rate limit, recusa do modelo, schema inválido e validação de evidência. Mensagens ao usuário explicam o que fazer sem revelar segredo ou stack trace. Logs registram código de erro, duração e identificador da execução, mas não guardam dados sensíveis ou chave da API.

Testes por propriedade conferem que taxa está entre zero e um, denominador zero gera estado explícito e filtros vazios não produzem comparação enganosa. Um teste de estado muda semestre e garante que a narrativa anterior seja removida. Um teste de mock

simula timeout e JSON inválido. Esses testes não substituem validação de modelo, mas impedem falhas comuns na aplicação.

Para implantação, Streamlit pode rodar localmente ou em serviço hospedado, mas o caminho de rede, armazenamento de segredo e política de sessão mudam. O segredo não fica no código nem em variável de frontend. O processo deve impor HTTPS, autenticação se houver dados protegidos, limite de consumo e atualização do aplicativo. O protótipo da aula usa dados fictícios e não precisa de login, conforme a atividade proposta.

A interface separa execução analítica de geração: o usuário pode atualizar estatísticas sem chamar a API e pedir narrativa explicitamente. Isso oferece controle de custo e reduz envio automático de contexto. A página informa modelo e modo de execução em termos compreensíveis; uma chave ausente ativa mock ou mensagem de configuração, não quebra o painel.

12.5 Roteiro da aula e decisão de publicação

19h–19h25: desenhamos arquitetura e pastas; 19h25–20h: construímos filtro, resumo e gráfico; 20h–20h30: testamos mudanças de estado e definimos contrato de narrativa.

20h30–21h: conectamos mock e validador; 21h–21h25: grupos configuram a integração API opcional usando segredo de ambiente e comparam com mock/local segundo dado, qualidade, custo e suporte; 21h25–21h50: testam timeout, amostra pequena, filtro divergente e ID inventado; 21h50–22h: demonstram dashboard e decisão.

O resultado entregue inclui aplicação, requirements, CSV sintético, resumo de dados, gráfico, adaptador, teste de validação e desenho de implantação. A chave individual não é requisito para demonstrar a arquitetura: o mock substitui o serviço, e a integração real usa o segredo configurado pelo professor para a prática.

Publicar requer autenticação, HTTPS, política de dados, restrições de uso, limites de chamadas e plano de atualização. Um protótipo da aula fica em ambiente de demonstração com dados fictícios.

Cada squad finaliza com uma decisão de publicação: somente notebook, aplicação local ou piloto restrito. A justificativa relaciona risco, público, dado e operação. Nenhum grupo usa dados pessoais reais sem autorização e revisão apropriada.

12.6 Implementar o adaptador e a fronteira de validação

O adaptador isola detalhes de transporte. Uma função interna recebe pergunta normalizada, evidências permitidas e formato de saída; cria a requisição do provedor; trata timeout e erro; e devolve conteúdo bruto junto com metadados mínimos de execução. O domínio não conhece endpoint, chave ou nome de SDK. Essa fronteira permite testar com mock, alternar modelo e manter cálculo independente da rede.

A mensagem de sistema/desenvolvedor deve ser curta e operacional: papel de narrador, evidências disponíveis, limites de inferência e schema esperado. A mensagem do usuário carrega pergunta e contexto permitido. O modelo não recebe acesso ao DataFrame original por padrão; a aplicação calcula métricas e oferece somente os agregados necessários. Esse desenho reduz a superfície de exposição e mantém números reprodutíveis localmente.

Uma resposta pode ser JSON com observação, caveat, próximo teste e lista de IDs. O parser valida campos obrigatórios, tipos, tamanho, IDs existentes e menções numéricas.

Se a saída vier como texto fora do schema, a aplicação não tenta corrigir silenciosamente com heurística frágil: pede nova resposta limitada ou usa fallback. A interface informa que a narrativa gerada não substitui a tabela calculada.

As falhas são tratadas na camada certa. Arquivo ausente ou coluna inválida é erro de ingestão; divisão por zero é condição analítica; API indisponível é transporte; ID inexistente é validação; texto que inventa causa é qualidade semântica. Mensagens ao usuário sugerem ação segura, enquanto logs técnicos registram código e duração sem guardar a chave ou dados sensíveis.

A integração real fica atrás de configuração explícita. O README explica instalar dependências, definir OPENAI_API_KEY no ambiente local ou secrets do host e escolher um modelo pela variável prevista. A equipe nunca imprime o segredo nem o armazena em arquivo versionado. A chave fornecida pelo professor é individualmente configurada e pode ser revogada; a atividade também funciona integralmente por mock.

O protótipo distingue cálculo de narrativa. Uma função pura calcula numerador, denominador e taxa, outra constrói visual, e uma terceira prepara pacote de evidências. Uma chamada remota só ocorre quando o estudante aciona gerar narrativa. Assim, o grupo pode apresentar uma EDA correta sem custo de API, comparar o texto gerado e reproduzir resultados em sessão nova.

Para avaliar se a LLM acrescenta valor, preparamos pares de respostas: template factual com os mesmos valores e rascunho gerado. Revisores comparam clareza, suporte, ressalvas, consistência e tempo de edição. Fluência não conta como precisão. Se o template é mais claro e seguro para uma tarefa repetitiva, o software pode usar template; a tecnologia é escolhida por benefício demonstrado.

Antes de publicar, o app precisa de revisão de identidade, transporte, retenção, limites de custo, monitoração e finalidade. A versão de sala não autentica usuários porque usa dados sintéticos e roda em ambiente controlado, conforme escopo didático; isso não autoriza reutilizá-la com dados reais. README marca claramente que o protótipo é educacional e lista controles ainda exigidos para ambiente institucional.

Uma demonstração de segurança usa uma cadeia de ataque simples: entrada tenta obter segredo, prompt pede coluna pessoal e texto do conjunto contém instrução adversarial. A resposta esperada é que segredo nunca esteja no contexto, coluna não esteja no pacote e conteúdo recuperado seja tratado como dado. O estudante aponta a validação no serviço e distingue prevenção de instrução textual.

O README descreve os limites de uso em linguagem direta: dados do exemplo são fictícios; a chave deve ficar local; o protótipo não autentica; não usar registros acadêmicos; e o app não toma decisão sobre pessoas. Essa documentação acompanha o código porque segurança e semântica também são parte de um software compreensível.

Ao final, cada componente possui uma pergunta de revisão: de onde veio o dado, qual função calculou a taxa, que estado criou o gráfico, que evidência entrou no prompt, quem conferiu a narrativa e como falha o provedor? Responder essas perguntas revela se a equipe entende o sistema completo ou apenas reproduziu a tela.

Síntese da trilha

Streamlit integra interface, EDA e narrativa, mas arquitetura continua necessária. O cálculo e o visual são determinísticos; o LLM produz um rascunho contextual; validação e revisão definem o que pode ser compartilhado.

A primeira metade começa no quadro com um diagrama em quatro camadas: inter-

face, domínio/EDA, adaptador de modelo e validação. Os grupos recebem um cenário e desenham o trajeto dos dados, identificam onde a chave vive e marcam o que pode sair do servidor. Em seguida, usam uma IA para contestar o diagrama e apresentam quais recomendações aceitaram ou rejeitaram após verificar riscos e contratos.

Na parte prática, o professor abre o protótipo sintético e muda filtro, denominador e tipo de pergunta. Alunos predizem a mudança antes de olhar o painel; depois verificam se tabela, gráfico e texto acompanham o mesmo estado. A demonstração evidencia que a tela atualizou não prova que cada componente foi recalculado com o mesmo contrato.

Cada grupo trabalha em ciclos pequenos: primeiro visual com cálculo local; depois narrativa por mock; em seguida validação de formato e evidência; por fim chamada opcional a serviço, se a chave e ambiente estiverem disponíveis. Essa ordem oferece caminho completo mesmo sem acesso à API. O aluno explica o ponto de integração em vez de depender de uma chave para comprovar conhecimento de arquitetura.

A tarefa para casa pede aplicativo mínimo, sem autenticação e com dados didáticos ou públicos aprovados. A entrega inclui código, README, fluxo arquitetural, uma resposta sustentada, um caso de recusa e evidência de que a chave não foi commitada. O professor fornece a chave ou demonstra secret local; estudante não publica a credencial nem usa dados reais de colegas.

Na aula seguinte, cada equipe apresenta pergunta, fonte, transformação, indicador, gráfico, narrativa, limite e próxima verificação em poucos minutos. Uma dupla revisora escolhe um valor e refaz o cálculo, depois muda um filtro para testar sincronização. Essa defesa individual explica contribuição e promove pontos por justificativa técnica, não somente por interface funcional.

A rubrica avalia correção da análise, separação de camadas, minimização de contexto, validação da saída, clareza do storytelling e resposta a falhas. Um mock é suficiente para demonstrar arquitetura, mas o estudante deve indicar como o adaptador de API seria configurado. Uma integração real não compensa métrica incorreta nem evidência ausente.

O artefato não é publicado externamente durante a atividade. A equipe registra requisitos antes de qualquer uso real: origem e autorização dos dados, identidade, política de retenção, custo, logs, responsável de negócio e processo de contestação. A conclusão prepara os capítulos de agentes e avaliação, nos quais geração deixa de ser uma chamada única e passa a participar de um fluxo controlado.

Uma política de saída define o que fazer com conteúdo bloqueado. A aplicação pode apresentar fatos e esconder apenas a narrativa; pode oferecer uma frase padrão sem inferência; ou pedir revisão. Para dados sintéticos da aula, o fallback pode ser um template com valores calculados. A interface deixa claro qual caminho foi usado, não imita uma resposta gerada quando a API falha.

O adaptador inclui identificador de correlação e tempo limite. Esse identificador conecta logs da aplicação sem conter pergunta ou segredo e facilita saber se a falha veio de timeout, limite do provedor ou resposta malformada. Métricas operacionais incluem latência, status e tamanho agregado de entrada/saída, coletadas com política de retenção.

O contrato de integração também estabelece cancelamento e idempotência para evitar que um clique repetido crie chamadas concorrentes. A interface desabilita o botão enquanto uma execução está ativa ou associa requisição a identificador; a camada de serviço impede duplicação e registra se a resposta corresponde ao estado atual da tela.

O schema da resposta restringe não apenas formato, mas também afirmações permitidas. Uma observação pode citar evidência; uma hipótese precisa usar linguagem

condicional; recomendação pode ser desativada no MVP. Campos obrigatórios impedem resumo sem caveat quando o grupo é pequeno. Regras no renderer escolhem o que aparece de acordo com os dados e a configuração da atividade.

Uma auditoria simples compara o texto original da API com o texto renderizado após validação. Diferenças são registradas como sentenças removidas ou campos rejeitados, permitindo entender se validação está funcionando. A equipe evita salvar payload completo quando ele contém dado sensível; usa amostras sintéticas e controles de acesso para depurar.

A turma pode representar a integração em pseudocódigo no quadro: validar entrada, consultar dados, calcular evidência, montar mensagens, chamar API, parsear saída, validar IDs e renderizar. Cada etapa tem erro possível e tratamento. O desenho deixa claro que uma chave habilita transporte, não concede ao modelo acesso irrestrito ao sistema.

Uma versão mínima pode ainda guardar metadados de modelo, versão do schema e hash do pacote de evidências no resultado de sessão. Isso não precisa expor prompt nem dado; serve para ligar narrativa aos cálculos que a produziram. Se o painel atualizar, o hash muda e a narrativa antiga não pode ser exibida como correspondente ao novo conjunto.

A aplicação educacional também documenta limites de versão e dependências. Atualização de SDK pode mudar formato de resposta, portanto adaptador tem verificação manual com mock antes de trocar versão. A escolha de modelo pode alterar estilo, latência e saída; uma pergunta de regressão confirma que o schema e as alegações verificáveis continuam corretos.

Uma futura evolução poderia adicionar saída estruturada nativa ou chamada de ferramenta; ela ainda passa pela validação da aplicação. O MVP usa uma única tarefa narrativa e não executa código arbitrário. Esse escopo reduz risco e mantém o objetivo da aula: compreender integração completa e fronteiras, não montar plataforma agentic inteira em uma noite.

A revisão humana continua parte do produto. O estudante confere valores, seleciona audiência e aprova compartilhamento. Uma interface pode exigir checkbox de confirmação para exportar, mas a confirmação não corrige erro; controles técnicos continuam necessários. O protótipo ensina que human in the loop só funciona quando a pessoa vê evidência suficiente para realmente revisar.

Referências de implementação

- Streamlit: caching and state management.
- OpenAI: Responses API, function calling e Structured Outputs (documentação consultada em 20 out. 2026).
- Python Software Foundation: venv e ambientes virtuais.